

Lightning Web Components

Introduction

Sheetal Chougule





Introduction to Lightning Web Components?



What is Lightning Component Framework?



- Component based framework enabled to build responsive and dynamic web applications
- Introduced in 2014 and built on modern tech stack like – HTML5, CSS, Javascript
- Provides range of pre-built components ready to use, customizable and allowing to build complex UI elements with minimal coding.
- provides a set of tools and services that enable developers to build, test, and deploy their applications
- The Lightning App Builder, provides a drag-and-drop interface for building Uis
- The Lightning Design System, which provides a set of UI guidelines and templates to ensure a consistent look and feel across applications.



Why is Lightning Web Components Faster than Lightning Components



Out-of-the box Lightning features to help increase productivity

- 1 Built using **Native Web Standards** like HTML, CSS, JS. Can take advantage of browser optimizations and performance improvements that have been developed over the years
- 2 **Lightweight Framework**, can be loaded and executed more quickly than Lightning Components
- 3 **Server-Side Rendering**, result in a faster initial load time and a better user experience.
- 4 **Smaller Bundle Size**, So can be downloaded and loaded more quickly by the browser
- 5 **Better Caching**, allowing components to be cached more effectively and reducing the amount of data that needs to be downloaded and processed by the browser.

ASTRO'S QUICK TIP:

*LWC is modern way of
development*





Aura And LWC – Coexistence & Interoperability



Salesforce Lightning Components vs Lightning Web Components

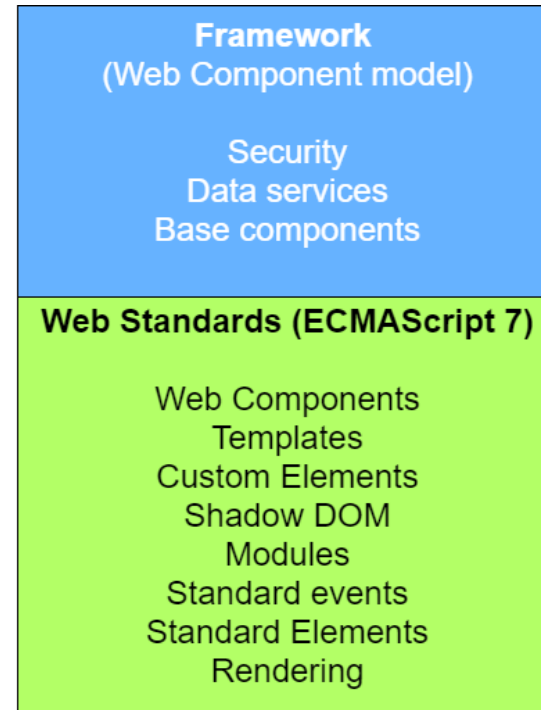
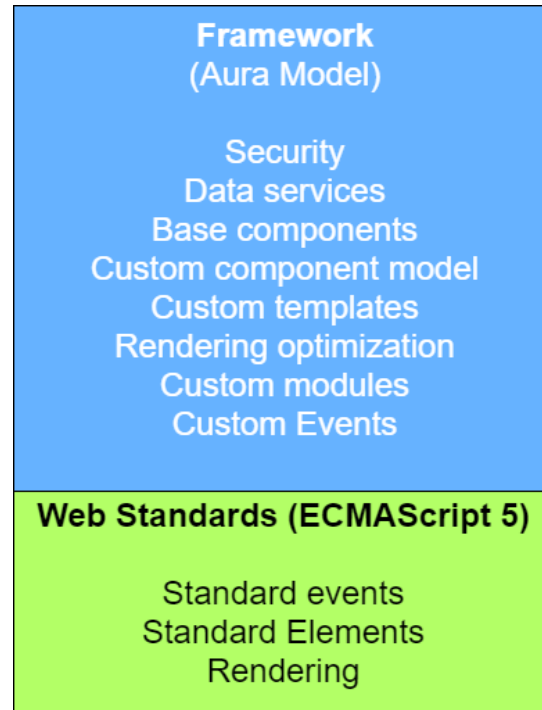


Feature	Lightning Components	Lightning Web Components
Introduced	2014 (Salesforce1)	2019
Programming Model	Custom Framework (Aura)	Native Web Standards (ECMAScript 6/7, Web Components)
Component Structure	Bundle (Component, Controller, Helper, Renderer, Style)	Module (HTML, JS, Meta XML, Optional CSS/Tests)
UI Template Language	<aura:component> + proprietary tags	Standard HTML5 + custom tags
Tooling	Requires Dev Console or Developer Console	Supports local development with Salesforce CLI + VS Code
Learning Curve	Steeper due to custom syntax and multiple files	Easier with modern JS and HTML
Reusability	Moderate – strong coupling between files	High – self-contained and modular
Rendering Engine	Framework-controlled rendering	Native browser rendering (Shadow DOM)
Performance	Heavier, slower in large apps	Lightweight, faster due to standard use
Testing Support	Limited and complex	Jest for unit testing, more standard tooling

Web Stack Transformation



- Create modular dynamic components
- Creation of templates, data binding, custom event system and services specific to the Salesforce platform
- Components and Framework doing all heavy lifting
- Developers need specific skillset



- Uses modern open web standards based on ECMAScript 7
- rely more and more on browser capabilities based on standards
- common component and programming model shared with other modern day frameworks such as Angular, React
- Skills acquired by developing Lightning Web Components can be interchanged to the other frameworks





Features Comparison

LWC is faster, optimized and future. Aura can co-exist with LWC.

Aura	LWC
Framework-heavy, proprietary	Standards-based, browser-native
Useful for legacy components	Recommended for all new Salesforce development
More complex structure	Cleaner, modular architecture
Application and component events	Native event bubbling and composition



Component Breakdown

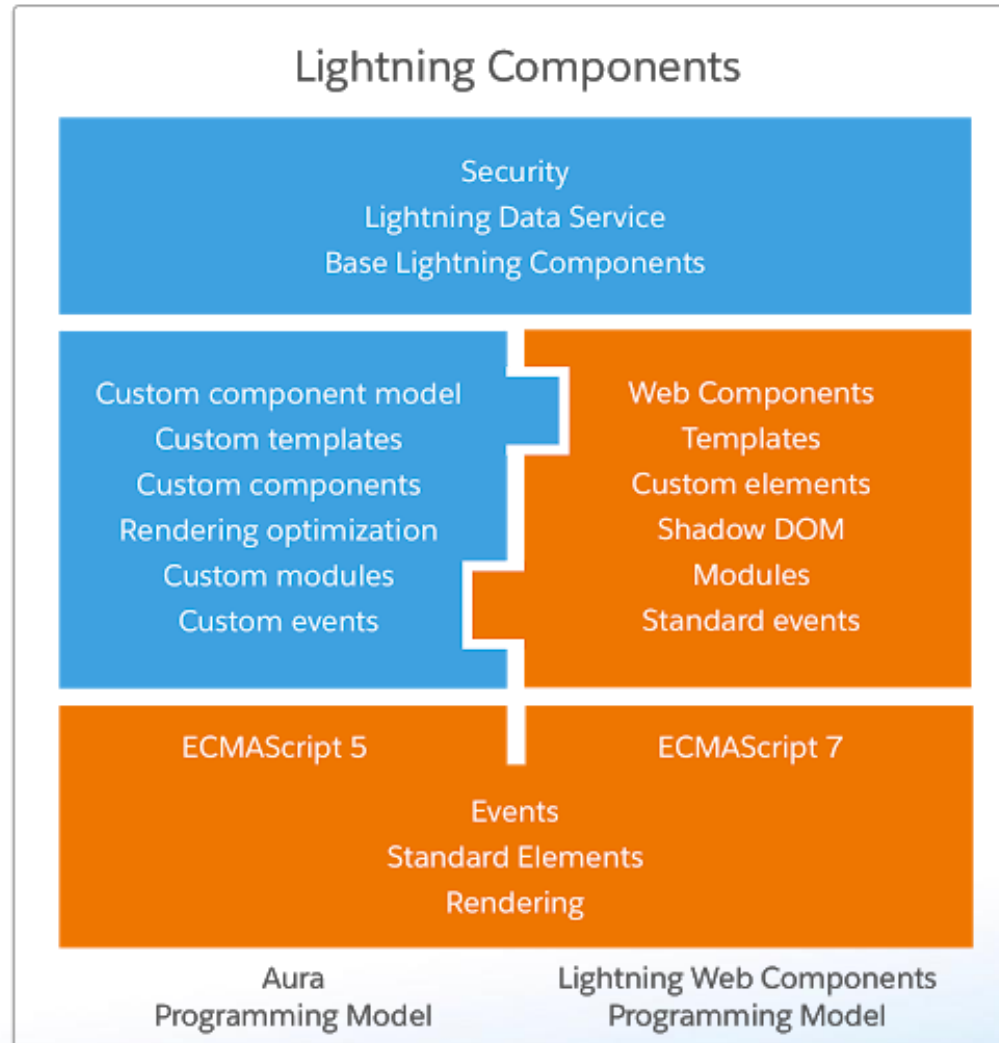


Aspect	Aura	LWC
Component Structure	Bundle (Component, Controller, Helper, Renderer, Style)	Module (HTML, JS, Meta XML, Optional CSS/Tests)
UI Template Language	<aura:component> + proprietary tags	Standard HTML5 + custom tags
Data Binding	Two-way ({!} and {}})	One-way ({}) + manual updates via JS
Event Handling	Component, Application Events	Custom DOM Events (Standard)
Shadow DOM	✗ Not supported	✓ Uses Shadow DOM
Decorator Support	✗ No decorators	✓ Supports @api, @track, @wire
Services Integration	Salesforce-specific Services	Lightweight access to same services via wire/service APIs
Dependency Management	Custom loader	ES6 module import/export
Scoped Styling (CSS)	Global styles via .cmp or .design files	Scoped styles in .css file with Shadow DOM isolation

How Aura and LWC Fit together?



Aura Framework



Shared Services

LWC Framework

**Common Web
Technology Stack**

Demo

Let's create our first LWC Component

salesforce



Use Case



We want to create a component where a user can type their name and see a greeting message.

Purpose:

- Understand how data flows from input to display.
- Know how to bind JavaScript properties to HTML.
- See how changes to input reflect live in the UI.



Component Overview



[helloUser](#)

File	Purpose
helloUser.html	Display the UI with elements
helloUser.js	Handle different data actions and events
helloUser.js-meta.xml	This file exposes your component to the Lightning App Builder.

Concepts Used



Concept	Explanation
LightningElement	Base class for all LWCs
@track	Reactive state, good for simple values
Data binding	{ } in HTML reflects data binding
Event Handling	JS Functions handle DOM events
Component Exposure	isExposed, targets in meta file

How it can be launched?

Aura page we created can now be launched using multiple ways.

1. Go to **App Builder** → Create a new **App Page**.
2. Drag the **helloUser** component from custom section.
3. Save and activate the page.
4. Preview the page – Type your name and see the greeting change live.



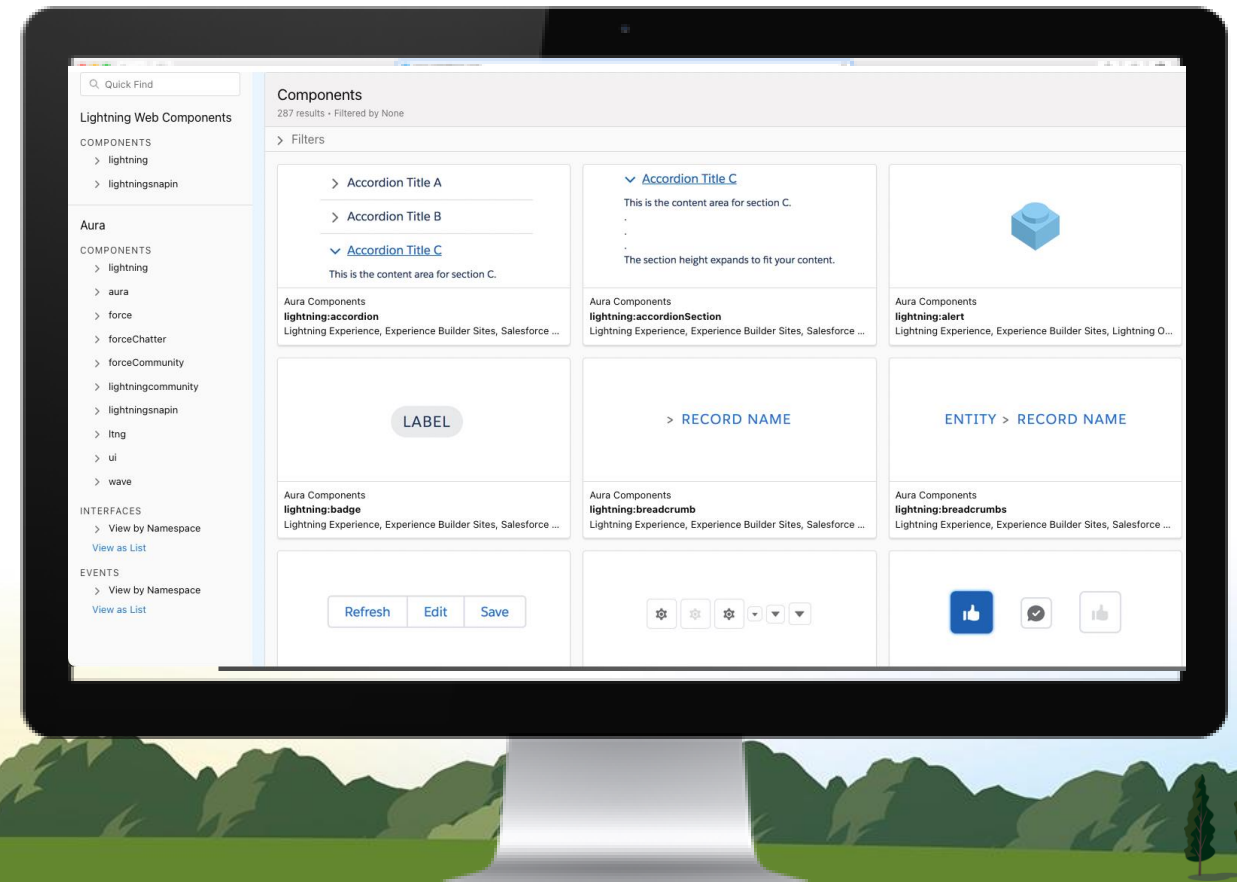
Component Library



Component Library



- The LWC Component Library is Salesforce's official collection of **prebuilt base Lightning components**.
- These are open-source, reusable UI elements built with Lightning Web Components.
- It includes layout components, inputs, buttons, modals, datatables, etc.
- Each component page includes:
 - Description
 - Example Preview
 - Markup (HTML code)
 - JavaScript controller (if required)
 - Attributes table
 - Events table



Why Use Component Library?

Re-use rather than Re-Invent

- Reduces development time
- Ensures design consistency with Salesforce Lightning Design System (SLDS)
- Improves performance and accessibility
- Salesforce-supported and updated

Demo

Using Base Component

salesforce

salesforce

References

- [Component library](#)
- [Playground to Test](#)
- [LWC Dev Guide](#)



Rhea Sy

Lead Developer, 21st Century Fox

Nithin Angri

Program Manager, AWS



thank you

Family Reunion

